

Roadmap 8 Tuần Chi Tiết — AI Engineer (Ứng Dụng)

Cách dùng file này: Mỗi mục kiến thức có 4 phần — (1) vì sao học, (2) yêu cầu cụ thể phải nắm được, (3) bài thực hành áp dụng vào project thật, (4) cách tự kiểm tra đã "xong" hay chưa. Chỉ chuyển sang mục tiếp theo khi trả lời được hết phần (2) mà không cần xem lại tài liệu.

TUẦN 1-2: Re-ranking, Query Expansion, Reciprocal Rank Fusion (RRF)

1.1 Cross-Encoder vs Bi-Encoder

Vì sao học: Đây là nền tảng của re-ranking — bước quan trọng nhất còn thiếu trong pipeline hybrid search hiện tại.

Yêu cầu phải nắm được:

- Giải thích được vì sao Bi-Encoder (dùng để retrieval ban đầu) nhanh nhưng kém chính xác hơn Cross-Encoder
- Giải thích được vì sao Cross-Encoder chính xác hơn nhưng không thể dùng cho toàn bộ dataset (chậm, không thể index trước)
- Vẽ được sơ đồ luồng: query → Bi-Encoder retrieval (top-k lớn, ví dụ top 50) → Cross-Encoder re-rank (chọn top-n nhỏ, ví dụ top 5) → đưa vào context LLM
- Biết tên ít nhất 1 model Cross-Encoder cụ thể dùng được cho tiếng Việt (ví dụ multilingual cross-encoder trên Hugging Face)

Thực hành: Thêm 1 bước re-rank vào pipeline retrieval hiện tại — lấy top 20 kết quả từ Qdrant, chạy qua Cross-Encoder, giữ lại top 5.

Tự kiểm tra: Có thể lấy 1 câu hỏi cụ thể, so sánh thứ tự kết quả TRƯỚC và SAU khi có re-rank, chỉ ra bằng ví dụ cụ thể tại sao kết quả sau tốt hơn.

1.2 Reciprocal Rank Fusion (RRF)

Vì sao học: Đây chính là cơ chế để merge kết quả dense search (semantic) + sparse search (BM25/keyword) — giải quyết trực tiếp bug hybrid search đang chạy dense-only.

Yêu cầu phải nắm được:

- Viết được công thức RRF: $score = \sum 1 / (k + rank_i)$ cho mỗi document xuất hiện trong nhiều danh sách kết quả
- Giải thích được vì sao RRF không cần chuẩn hóa điểm số giữa 2 hệ thống retrieval khác nhau (dense score và BM25 score có scale khác nhau, khó so sánh trực tiếp) — đây là lý do RRF được chọn thay vì cộng điểm trực tiếp
- Giải thích được vai trò của hằng số k (thường = 60) trong công thức
- Biết Qdrant hỗ trợ RRF native qua Query API (không cần tự code từ đầu)

Thực hành: Sau khi fix xong bug BM25 sparse embedding, implement RRF để merge kết quả dense + sparse trong pipeline hiện tại. So sánh kết quả trước (dense-only) và sau (hybrid + RRF).

Tự kiểm tra: Có thể tự tay tính RRF score cho 1 ví dụ nhỏ (3-4 documents, 2 danh sách rank) trên giấy, không cần code.

1.3 Query Expansion

Vì sao học: Câu hỏi ngắn/thiếu ngữ cảnh của người dùng (đặc biệt tiếng Việt, viết tắt, thiếu dấu) khiến retrieval kém — query expansion giúp retrieval tốt hơn trước khi search.

Yêu cầu phải nắm được:

- Phân biệt được 2 kiểu: (a) mở rộng bằng LLM (rewrite câu hỏi thành nhiều cách diễn đạt) và (b) mở rộng bằng đồng nghĩa/thuật ngữ liên quan
- Giải thích được rủi ro: query expansion có thể làm "loãng" ý định gốc của câu hỏi nếu mở rộng quá nhiều
- Biết được HyDE (Hypothetical Document Embeddings) là gì và khác gì với query expansion thông thường

Thực hành: Kiểm tra lại logic query rewriting đang có trong retry loop (bug rewritten query không được dùng) — sau khi fix bug, đánh giá xem query rewriting hiện tại có đang làm tốt việc expansion không, hay chỉ đơn giản là rephrase.

Tự kiểm tra: Lấy 3 câu hỏi tiếng Việt thiếu dấu/viết tắt thực tế từ log hệ thống, viết tay ra query đã "expand" đúng cách và giải thích vì sao expand như vậy sẽ cải thiện retrieval.

TUẦN 3-4: RAGAS Evaluation + Guardrails cơ bản

2.1 RAGAS Metrics

Vì sao học: Thay "test tay" bằng số liệu đo được — trực tiếp phục vụ công việc test/sửa lỗi hiện tại, và tạo bằng chứng định lượng cho CV.

Yêu cầu phải nắm được:

- Giải thích được **Faithfulness**: đo việc câu trả lời của bot có bám sát vào context được retrieve hay đang "bịa" (hallucination)
- Giải thích được **Answer Relevance**: đo việc câu trả lời có thực sự trả lời đúng câu hỏi hay không (khác với Faithfulness — 1 câu trả lời có thể faithful với context nhưng vẫn lạc đề)
- Giải thích được **Context Precision** và **Context Recall**: đo chất lượng của bước retrieval (có lấy đúng và đủ document liên quan không)
- Biết mỗi metric cần input gì (question, context, answer, ground truth) — Context Recall cần ground truth, còn Faithfulness thì không

Thực hành: Chuẩn bị 1 bộ 15-20 câu hỏi test (song ngữ Việt-Anh) đại diện cho các loại câu hỏi thật trong hệ thống. Chạy RAGAS trên bộ này TRƯỚC khi áp RRF/re-ranking, ghi lại số liệu baseline.

Tự kiểm tra: Có bảng số liệu cụ thể (không phải ước lượng) cho 4 metrics trên, và giải thích được vì sao 1 câu trả lời cụ thể trong bộ test bị điểm Faithfulness thấp.

2.2 Đo lại sau khi cải tiến (đóng vòng feedback loop)

Yêu cầu phải nắm được:

- Hiểu vì sao phải dùng CÙNG bộ câu hỏi test để so sánh trước/sau (kiểm soát biến, tránh so sánh sai)
- Biết cách trình bày kết quả so sánh dạng before/after có số liệu — đây là format dùng để viết vào CV/case study

Thực hành: Sau khi có RRF + re-ranking từ tuần 1-2, chạy lại RAGAS trên đúng bộ câu hỏi test tuần trước. Ghi lại % cải thiện của từng metric.

Tự kiểm tra: Có 1 bảng so sánh before/after với số liệu cụ thể (ví dụ: Faithfulness từ 0.65 → 0.82). Đây sẽ là nội dung chính của case study tuần 8.

2.3 Guardrails cơ bản

Vì sao học: Chatbot production cần chống bị dẫn dụ trả lời sai/nhạy cảm — khác với chatbot đồ án chỉ cần chạy đúng use-case chuẩn.

Yêu cầu phải nắm được:

- Phân biệt được guardrail ở **input** (chặn/lọc câu hỏi độc hại trước khi vào LLM) và **output** (kiểm tra câu trả lời trước khi trả về user)
- Biết ít nhất 1 công cụ cụ thể (NeMo Guardrails hoặc tự viết rule-based filter đơn giản bằng regex/keyword cho tiếng Việt)
- Hiểu được giới hạn: guardrail rule-based dễ bị lách, guardrail bằng LLM riêng (LLM-as-judge) chính xác hơn nhưng tốn thêm 1 lần gọi model (tăng latency + chi phí)

Thực hành: Viết 5-10 test case "cố tình hỏi lệch hướng" (ví dụ hỏi ngoài phạm vi domain, cố gắng lấy system prompt) và kiểm tra bot hiện tại phản ứng thế nào.

Tự kiểm tra: Liệt kê được ít nhất 3 lỗ hổng cụ thể phát hiện được trong bot hiện tại qua bài test trên.

TUẦN 5-6: FastAPI + Docker

3.1 FastAPI

Vì sao học: Nếu tích hợp voice-mode hoặc bất kỳ tính năng AI mới vào web, cần API riêng để giao tiếp với frontend.

Yêu cầu phải nắm được:

- Viết được 1 endpoint POST nhận JSON input, gọi hàm xử lý (RAG pipeline hoặc gọi LLM), trả JSON output
- Hiểu và dùng được **Pydantic model** để validate input/output (không nhận request sai format)
- Hiểu sự khác biệt giữa **sync và async endpoint** trong FastAPI — vì sao cần async khi gọi LLM API (I/O-bound, chờ network)
- Biết cách xử lý **streaming response** (trả từng phần câu trả lời về, giống hiệu ứng ChatGPT đang gõ) — dùng `StreamingResponse`

Thực hành: Viết 1 API endpoint nhận câu hỏi, chạy qua RAG pipeline hiện tại, trả kết quả kèm sources. Test bằng cách gọi thử qua Postman hoặc curl.

Tự kiểm tra: API chạy được, trả đúng response cho ít nhất 3 câu hỏi test, và có xử lý lỗi cơ bản (ví dụ input rỗng, timeout khi gọi LLM).

3.2 Docker

Vì sao học: Đóng gói môi trường để chạy nhất quán trên server, tránh lỗi "chạy máy tôi thì được".

Yêu cầu phải nắm được:

- Viết được Dockerfile cơ bản: base image Python, copy code, install dependencies, expose port, chạy uvicorn
- Hiểu sự khác biệt giữa **image** và **container** (image là bản đóng gói tĩnh, container là instance đang chạy)
- Biết cách dùng **docker-compose** để chạy nhiều service cùng lúc (ví dụ API + Qdrant + Redis trong 1 file compose)
- Hiểu vì sao cần `.dockerignore` (tránh copy file không cần thiết như `venv/`, `.git/` vào image, làm image nặng)

Thực hành: Viết Dockerfile cho API ở mục 3.1, build image, chạy container, kiểm tra API hoạt động đúng bên trong container. Nếu hệ thống hiện tại dùng Qdrant qua Docker rồi, thử viết docker-compose gộp API + Qdrant vào 1 file.

Tự kiểm tra: Build image thành công không lỗi, container chạy ổn định, gọi được API từ ngoài container vào đúng port đã expose.

TUẦN 7: Model Serving cơ bản + Multi-agent Orchestration

4.1 Serving cơ bản (Ollama/vLLM ở mức khái niệm + thực hành nhẹ)

Vì sao học: Đủ để hiểu và nói chuyện trong interview, không cần chuyên sâu benchmark.

Yêu cầu phải nắm được:

- Giải thích được **Time-to-first-token (TTFT)** là gì và vì sao nó quan trọng với UX (người dùng cảm nhận độ trễ ra sao)
- Giải thích được **throughput** (số request/token xử lý được mỗi giây) khác TTFT thế nào — đánh đổi giữa 2 chỉ số này khi tăng batch size
- Biết chạy 1 model nhỏ bằng Ollama trên máy local (ví dụ Qwen 7B hoặc tương đương), gọi qua API local
- Biết khái niệm **quantization** ở mức cơ bản: giảm độ chính xác số (FP16 → INT8/INT4) để giảm dung lượng model và tăng tốc, đổi lại giảm nhẹ độ chính xác — không cần tự benchmark GGUF/AWQ chi tiết

Thực hành: Cài Ollama, chạy 1 model nhỏ, so sánh tốc độ trả lời với việc gọi API cloud (OpenAI/Gemini) cho cùng 1 câu hỏi.

Tự kiểm tra: Có thể giải thích bằng lời (không cần đọc lại tài liệu) sự khác nhau giữa chạy model local qua Ollama và gọi API cloud — về chi phí, độ trễ, quyền kiểm soát dữ liệu.

4.2 Multi-agent Orchestration trong LangGraph

Vì sao học: Mở rộng từ LangGraph đơn agent (đang dùng) sang nhiều agent phối hợp — kỹ năng JD nhấn mạnh.

Yêu cầu phải nắm được:

- Giải thích được khi nào nên tách thành nhiều agent (mỗi agent chuyên 1 nhiệm vụ, ví dụ 1 agent retrieval + 1 agent tổng hợp câu trả lời) thay vì 1 agent làm hết
- Vẽ được sơ đồ state graph có ít nhất 1 **cycle** (vòng lặp, ví dụ agent tự kiểm tra câu trả lời và quay lại retrieval nếu chưa đủ thông tin) — liên hệ trực tiếp đến bug retry loop đã gặp
- Giải thích được khái niệm **persistent state** trong LangGraph: state được giữ và truyền qua các node thế nào, vì sao bug "rewritten query không được dùng lại" chính là lỗi quản lý state

Thực hành: Vẽ lại sơ đồ graph hiện tại của chatbot, đánh dấu rõ node nào đọc/ghi state nào — dùng để phát hiện các bug tương tự bug retry loop trong tương lai.

Tự kiểm tra: Có thể chỉ vào sơ đồ và giải thích chính xác luồng state đi qua từng node, không cần đọc code.

TUẦN 8: Portfolio + Technical English + Monitoring nhẹ

5.1 Trace logging / Observability cơ bản

Vì sao học: JD yêu cầu phát hiện lỗi production real-time — bạn đang tự debug lỗi nên đây là công cụ trực tiếp giúp việc đó.

Yêu cầu phải nắm được:

- Biết log tối thiểu cần có cho mỗi query: câu hỏi gốc, câu query đã rewrite, danh sách document retrieve được (kèm score), câu trả lời cuối, thời gian xử lý mỗi bước
- Biết ít nhất 1 công cụ cụ thể (Langfuse hoặc LangSmith — cả hai có bản free/self-host) để xem lại trace của 1 request

Thực hành: Setup 1 trong 2 công cụ trên, log lại pipeline hiện tại, dùng để tìm 1 lỗi thật (ví dụ query nào bị retrieve sai) qua giao diện trace thay vì đọc log text.

Tự kiểm tra: Có thể mở giao diện trace, tìm 1 request cụ thể, và đọc được toàn bộ luồng xử lý của nó.

5.2 Case Study bằng tiếng Anh

Vì sao học: JD yêu cầu rõ khả năng giao tiếp kỹ thuật tiếng Anh — điểm khác biệt để đạt nhất so với ứng viên khác.

Yêu cầu phải nắm được:

- Viết được case study theo cấu trúc: **Problem** → **Investigation** → **Fix** → **Result (số liệu)** — không viết dạng kể chuyện, viết dạng kỹ thuật ngắn gọn
- Dùng đúng thuật ngữ tiếng Anh đã học (Faithfulness, Hybrid Search, RRF, TTFT...) một cách tự nhiên trong câu, không dịch word-by-word từ tiếng Việt

Thực hành: Viết 1 case study hoàn chỉnh (300-500 từ) về bug BM25 sparse embedding: mô tả vấn đề, cách phát hiện, cách fix, số liệu RAGAS trước/sau.

Tự kiểm tra: Đọc lại case study, tự hỏi: nếu người đọc là 1 senior engineer chưa biết gì về hệ thống của mình, họ có hiểu được vấn đề và đánh giá được năng lực qua đoạn viết này không?

Bảng tổng kết Deliverables sau 8 tuần

Tuần	Deliverable cụ thể
1-2	Pipeline hybrid search có re-ranking + RRF, so sánh kết quả trước/sau bằng ví dụ thật
3-4	Bộ test RAGAS 15-20 câu, số liệu before/after, danh sách lỗi hổng guardrail phát hiện được
5-6	1 API FastAPI đóng gói bằng Docker, chạy được từ ngoài container
7	Sơ đồ state graph LangGraph có annotation luồng state, hiểu TTFT/throughput/quantization ở mức giải thích được
8	1 case study tiếng Anh hoàn chỉnh + hệ thống trace logging đang chạy

Nguyên tắc xuyên suốt: Không chuyển sang mục mới nếu chưa trả lời được phần "Yêu cầu phải nắm được" mà không cần mở lại tài liệu. Nếu 1 tuần không đủ thời gian hoàn thành, ưu tiên chất lượng hiểu sâu 1 phần hơn là lướt qua tất cả.